Exercise 11: Recurrent Neural Networks (RNNs)

Task 1: Questions about Recurrent Neural Networks

Answer the following questions:

a) What is the main difference between feedforward neural networks and recurrent neural networks regarding its network structure?
b) What role does the hidden state play in an RNN, and how does it enable sequential data processing?
c) What is the vanishing gradient problem in RNNs, and why does it occur?

Task 2: Determining the number of trainable parameters

Calculate the number of trainable parameters given the information about the dimensions of the input, the output and/or the hidden states.

a) $m = 5, n = 32, o = 10$
b) $n = 10, Number\ of\ weights\ in\ W_{xh} = 40, o = 3$
c) $W_{hh} = \begin{bmatrix} 1 & 0.5 & 1 & 1 & 0.4 \\ 0.1 & 0.6 & 0 & 1 & 0 \\ 0.8 & 0.5 & 0.1 & 0 & 1 \\ 0.9 & 0.2 & 0 & 0 & 0.2 \\ 0.1 & 1.0 & 0.3 & 0 & 0.9 \end{bmatrix}, m = 8, o = 2$

Task 3: Calculating the results of RNNs

Calculate the results y for each time step of the forward pass of the given Elman net.

Note on the activation function: *Typically, a classic Elman network utilizes the hyperbolic tangent function* ($\tanh$) *as the activation function for the hidden state. However, to simplify the manual calculations here, we will use the **ReLU function*** ($f(z) = \max(0, z)$)*.*

Demonstration example:

Given:

$$x_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, x_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$h_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$W_{xh} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix},$$

$$W_{hh} = \begin{bmatrix} 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 0.5 \end{bmatrix}$$

$$b_h = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

$$W_{hy} = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix},$$

$$b_y = \begin{bmatrix} 0 \end{bmatrix}$$

Your task:

Inputs:

$$x_1 = \begin{bmatrix} 1 \\ 2 \\ 0 \\ -1 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 0 \\ -2 \\ 4 \\ 1 \end{bmatrix}$$

Initial hidden state:

$$h_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Weights and biases:

$$W_{xh} = \begin{bmatrix} 2 & -1 & 3 & 1 \\ 0 & 2 & -1 & -2 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 1 & 2 \\ -1 & 3 \end{bmatrix}, \quad b_h = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

$$W_{hy} = \begin{bmatrix} 2 & -1 \\ 1 & 1 \end{bmatrix}, \quad b_y = \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

Task 4: Implementing an RNN for the mnist dataset

Let's assume that we are implementing a document scanner. Thus, the input image is scanned line by line. We can interpret each line as one particular time step. Thus, we can train an RNN to predict the class of the mnist digit as early as possible. For this purpose, we need to generate an output y_t for each time step t, which is fed into classification layers (thus, at least a softmax layer is needed). Setting a threshold for the softmax score allows us to decide how "confident" the net needs to be to generate the output. To solve this task we perform the following steps:

- Subdivide each image into a sequence of 28 vectors of length 28
- Create a new training and test set by combining 1 up to 28 of these vectors to form sequences of image rows. Each sequence needs to be associated with the corresponding label (i.e., the class to predict).
- Build a network consisting of a single recurrent layer and a number of MLP layers (your decision!) with a softmax output, which are the actual classifier. Use the `TimeDistributed` layer class to make keras applying the same layers for each time step.
- Fit the model.
- Run an evaluation on the test data. This needs to be done slightly differently than in previous exercises: Set a confidence threshold so that the net returns the class as soon as its confidence meets or exceeds this threshold.
- Use the accuracy metric in both the model fitting and testing evaluations.

What is the average time step when the RNN makes the decision for a class?
Measure both the performance of the RNN model and the average time step when the RNN makes the decision dependent on the confidence threshold.
To get started you can use the code provided via Moodle.